AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Implementacja listy dwukierunkowej w C++ z zastosowaniem GitHub

Autor:
Jakub Pacułt

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Struktura i implementacja listy 3
1.2. Kod źródłowy węzła 3
1.3. Zarządzanie pamięcią i testowanie 4

2. Analiza problemu **5**

2.1. Zastosowanie struktur danych i wzorców projektowych 5
2.2. Sposób działania programu i algorytmów 6
2.3. Przykład działania algorytmu wstawiania 6
2.4. Opis narzędzi wspierających projekt 7

3. Projektowanie **8**

3.1. Środowisko i narzędzia programistyczne 8
3.2. Ustawienia kompilatora i konfiguracja projektu 8
3.3. Architektura systemu i diagram klas (UML) 9
3.4. Sposób użycia systemu kontroli wersji (Git) 10

4. Implementacja **11**

4.1. Omówienie wybranych fragmentów kodu źródłowego 11
4.1.1. Bezpieczne wstawianie elementu pod wskazany indeks 11
4.1.2. Implementacja Wzorca Iteratora 12
4.2. Opis uzyskanych wyników i działania programu 12

5. Wnioski **14**

Literatura **15**

Spis rysunków **16**

Spis tabel **17**

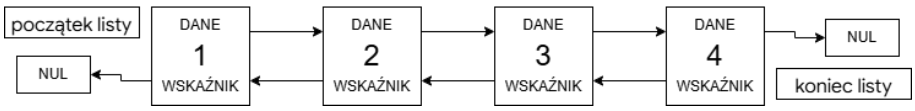
Spis listingów **18**

1. Ogólne określenie wymagań

Celem niniejszego projektu jest zaimplementowanie i przetestowanie struktury danych typu dwukierunkowa lista wiązana w języku C++. Oczekiwanym wynikiem jest stworzenie efektywnego kontenera dynamicznego, który umożliwia operacje wstawiania, usuwania oraz przeglądania elementów zarówno w kierunku rosnącym, jak i malejącym. Program ma na celu zaprezentowanie poprawnego zarządzania pamięcią dynamiczną oraz operacji na wskaźnikach.

1.1. Struktura i implementacja listy

Każdy element listy dwukierunkowej jest reprezentowany przez osobny węzeł. Jeśli chcemy odwołać się do literatury naukowej lub dokumentacji, wykonujemy to w ten sposób.



Rys. 1.1. Schemat powiązań w liście dwukierunkowej

Węzeł przechowuje wartość całkowitą oraz dwa wskaźniki: na poprzedni oraz następny element.
Węzeł początkowy posiada wskaźnik poprzednika ustawiony na wartość pustą (‘nullptr‘). Rysunek 1.1 (s. 3) pokazuje schemat powiązań między węzłami listy.

Tab. 1.1. Złożoność obliczeniowa operacji

Operacja	Złożoność czasowa	Złożoność pamięciowa
Dodanie na początek/koniec	$O(1)$	$O(1)$
Wyszukanie elementu	$O(n)$	$O(1)$
Usunięcie elementu	$O(n)$ lub $O(1)$	$O(1)$

Tabela 1.1 (s. 3) przedstawia zestawienie złożoności obliczeniowej dla podstawowych operacji na liście dwukierunkowej.

1.2. Kod źródłowy węzła

Kod 3 (s. 12) przedstawia implementację klasy pojedynczego węzła, która stanowi fundamentalny budulec struktury listy dwukierunkowej.

```
#ifndef NODE_H
#define NODE_H

class Node {
public:
    int data;     // Wartosc przechowywana w wezle
    Node* prev;   // Wskaznik na poprzedni wezel
    Node* next;   // Wskaznik na nastepny wezel

    // Konstruktor inicjalizujacy wezel
    Node(int val) : data(val), prev(nullptr), next(nullptr) {}
};

#endif
```

Listing 1. Definicja struktury węzła Node

1.3. Zarządzanie pamięcią i testowanie

Ponieważ lista dwukierunkowa alokuje pamięć dynamicznie przy każdym dodaniu nowego elementu za pomocą operatora ‘new‘, kluczowym elementem projektu jest poprawnie napisany destruktor oraz metoda czyszcząca.

Każde usunięcie elementu wymaga odpowiedniego przepięcia wskaźników ‘prev‘ oraz ‘next‘ sąsiadujących węzłów, a także zwolnienia pamięci operatorem ‘delete‘. Zapobiega to groźnym wyciekom pamięci (ang. *memory leaks*). Testowanie poprawności implementacji odbywa się poprzez sekwencję operacji w pliku głównym programu (‘main.cpp‘), gdzie sprawdzane są skrajne przypadki, takie jak operacje na pustej liście czy usuwanie jedynego elementu.

Następnym krokiem w procesie rozwoju oprogramowania jest kontrola wersji z użyciem systemu Git. Na rysunku 1.2 (s. 5) zaprezentowano przykładowy widok pomyślnego przejścia testów.

```
Dodawanie elemntow do listy:
Lista (przod): 5 10 20

Dodawania na danym indeksie:
Lista (przod): 5 15 10 20

Wyswietlanie wstecz:
Lista (tyl): 20 10 15 5

Usuwanie z przodu i z tylu:
Lista (przod): 15 10

Usuwanie na danym indeksie:
Stan przed usunieciem: Lista (przod): 15 10 30 40
Stan po usunieciu indeksu 1: Lista (przod): 15 30 40

Czyszczenie listy:
Lista (przod):
```

Rys. 1.2. Wynik działania testów listy (operacje wyświetlania, dodawania, usuwania)

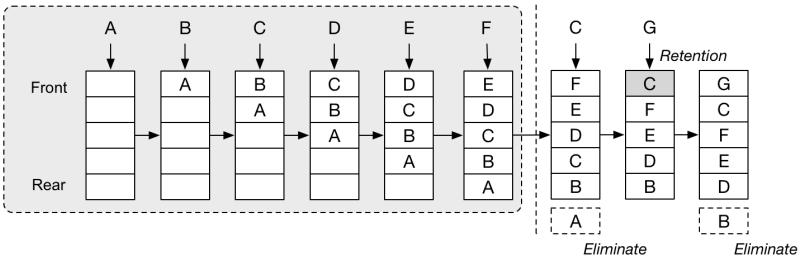
2. Analiza problemu

2.1. Zastosowanie struktur danych i wzorców projektowych

Dwukierunkowa lista wiązana jest jedną z fundamentalnych struktur danych w informatyce. Znajduje szerokie zastosowanie w sytuacjach wymagających częstych operacji wstawiania i usuwania elementów w dowolnym miejscu kolekcji, bez konieczności realokacji całej pamięci (co wyróżnia ją na tle zwykłych tablic). Jest powszechnie wykorzystywana m.in. w:

- Mechanizmach cofania operacji (*Undo/Redo*) w edytorach tekstów czy programach graficznych.
- Implementacji pamięci podręcznej typu LRU [1](*Least Recently Used*).
- Zarządzaniu oknami i historią nawigacji w przeglądarkach internetowych.

W ramach projektu zastosowano również wzorce projektowe: **Wzorzec Iterator** (ułatwiający sekwencyjny dostęp do elementów listy bez ujawniania jej wewnętrznej reprezentacji) oraz **Wzorzec Fabryki** (centralizujący proces tworzenia nowych obiektów węzłów).



Rys. 2.1. Przykładowy algorytm LRU

2.2. Sposób działania programu i algorytmów

Program zarządza dynamiczną alokacją pamięci za pomocą wskaźników łączących poszczególne węzły (`Node`). Każdy węzeł przechowuje wartość całkowitą oraz dwa wskaźniki: na element następny (`next`) oraz poprzedni (`prev`).

Główne algorytmy zaimplementowane w projekcie obejmują:

1. **Dodawanie elementów (`addFront`, `addBack`, `addAtIndex`):** Alokacja nowego węzła w pamięci za pomocą fabryki, a następnie odpowiednie przepięcie wskaźników "głowy" (`head`), "ogona" (`tail`) oraz sąsiadujących węzłów.
2. **Usuwanie elementów (`removeFront`, `removeBack`, `removeAtIndex`):** Wyszukanie elementu, zmiana powiązań wskaźników w celu pominięcia usuwanego węzła, zwolnienie pamięci funkcją `delete` oraz aktualizacja rozmiaru listy.
3. **Przechodzenie przez iteratory:** Klasa pomocnicza *ListIterator* zarządza aktualnym wskaźnikiem i przeciąża operatory inkrementacji (`++`), umożliwiając bezpieczne przeglądanie zawartości.

2.3. Przykład działania algorytmu wstawiania

Aby zilustrować działanie algorytmu wstawiania, przeanalizujmy ręcznie proces dodawania elementu o wartości 20 na sam początek listy, w której znajduje się już jeden element o wartości 10.

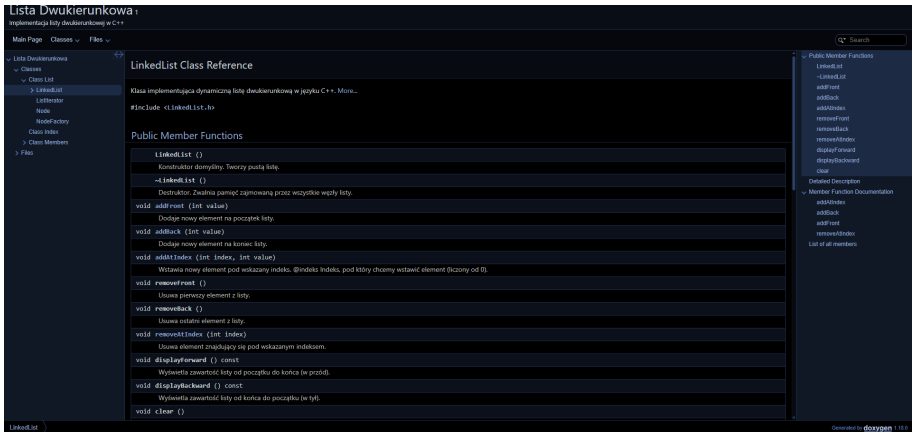
1. **Stan początkowy:** Lista posiada wskaźnik `head` skierowany na węzeł [10]. Węzeł [10] ma `prev = nullptr` oraz `next = nullptr`.
2. **Krok 1 (Utworzenie węzła):** Fabryka tworzy nowy węzeł o wartości 20 (`newNode`). Jego wskaźniki ustawione są domyślnie na `nullptr`.

3. **Krok 2 (Przepięcie wskaźników):**
   - Nowy węzeł wskazuje swoim polem `next` na dotychczasową głowę: `newNode->next = head;` (czyli węzeł [10]).
   - Dotychczasowa głowa aktualizuje wskaźnik wsteczny: `head->prev = newNode;` (węzeł [10] wskazuje na nowy węzeł).
   - Wskaźnik `head` całego obiektu listy zostaje zaktualizowany i wskazuje teraz na `newNode` ([20]).
4. **Stan końcowy:** Otrzymujemy spójną listę dwukierunkową: `head` $\leftrightarrow$ [20] $\leftrightarrow$ [10] $\leftrightarrow$ `tail`.

2.4. Opis narzędzi wspierających projekt

W procesie tworzenia i testowania oprogramowania wykorzystano nowoczesne narzędzia inżynieryjne:

- **Git oraz GitHub:**[2, 3]System kontroli wersji, który umożliwił śledzenie zmian w kodzie źródłowym, bezpieczne testowanie nowych funkcji na oddzielnych gałęziach (*branchach*) oraz archiwizację projektu.
- **Microsoft Visual Studio oraz MSBuild:** Środowisko programistyczne dostarczające kompilatora MSVC (narzędzie wiersza poleceń `cl`), służące do budowania i uruchomienia kodu napisanego w języku C++.
- **Doxygen:**[4] Narzędzie do automatycznego generowania dokumentacji technicznej na podstawie ustandaryzowanych komentarzy zawartych w kodzie źródłowym.



Rys. 2.2. Strona HTML wygenerowana przez program

3. Projektowanie

3.1. Środowisko i narzędzia programistyczne

Proces projektowania i implementacji aplikacji został oparty o nowoczesny stos technologiczny zapewniający wydajność, skalowalność oraz zgodność ze standardami inżynierii oprogramowania:

- **Język programowania:** C++ (w standardzie C++17/C++20), ze względu na pełną kontrolę nad zarządzaniem pamięcią oraz wsparcie dla paradygmatu obiektowego.
- **Środowisko programistyczne (IDE) i Kompilator:** Microsoft Visual Studio Community oraz natywny kompilator MSVC (`cl.exe`) wraz z narzędziem budującym `MSBuild`.
- **System kontroli wersji:** Git oraz platforma GitHub, służące do zarządzania historią kodu i pracy grupowej/eksperymentalnej.
- **Generator dokumentacji:** Doxygen, wspierający ustandaryzowany format komentarzy w kodzie źródłowym.
- **Dodatkowe biblioteki:** Standardowa biblioteka języka C++ (*STL*), w tym nagłówki podstawowe takie jak `<iostream>`.

3.2. Ustawienia kompilatora i konfiguracja projektu

Kompilacja kodu źródłowego wymaga specyficznych flag konfiguracyjnych w celu zapewnienia poprawności działania programu, zwłaszcza w zakresie obsługi mecha-

nizmów języka C++:

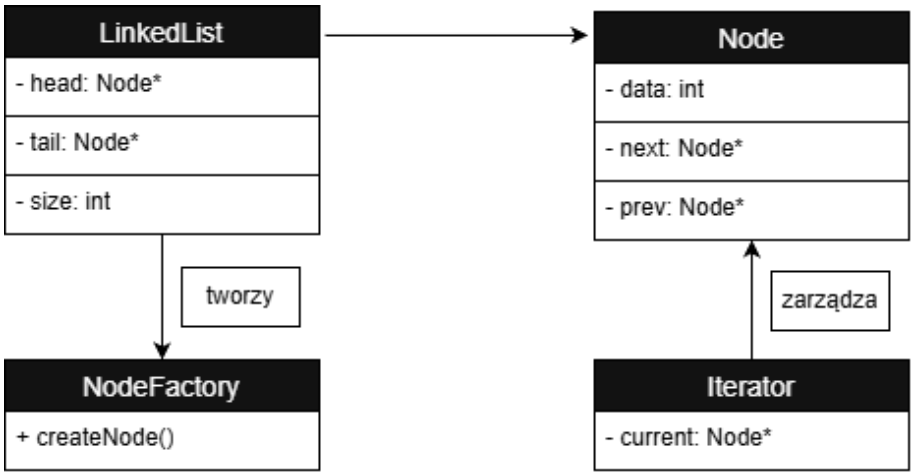
- **Flaga obsługi wyjątków (`/EHsc`):** Kluczowy parametr kompilatora MSVC, który włącza standardową obsługę wyjątków synchronicznych (*Exception Handling*). Umożliwia to poprawną konstrukcyjnie obsługę błędów alokacji pamięci oraz działania obiektów.
- **Architektura docelowa:** Projekt skonfigurowany jest dla architektury 64-bitowej (`x64`), co pozwala na pełne wykorzystanie przestrzeni adresowej nowoczesnych procesorów.
- **Konfiguracje budowania:** Wyróżnia się dwa główne warianty:
  - *Debug:* Zawiera symbole debugowania, wyłączone optymalizacje, przeznaczony do wykrywania błędów.
  - *Release:* Zoptymalizowany pod kątem wydajności i szybkości działania programu, z pominięciem metadanych diagnostycznych.

3.3. Architektura systemu i diagram klas (UML)

Projekt zrealizowano w oparciu o cztery główne komponenty powiązane relacjami obiektowymi oraz wzorcami projektowymi:

1. **Klasa `Node`:** Reprezentuje pojedynczy węzeł listy. Zawiera pole danych (`int data`) oraz wskaźniki na sąsiadów (`Node* next`, `Node* prev`).
2. **Klasa `LinkedList`:** Główna struktura zarządzająca kolekcją. Przechowuje wskaźniki na początek (`head`) i koniec (`ogony/tail`) oraz aktualny rozmiar (`size`).
3. **Klasa `ListIterator` (Wzorzec Iterator):** Odpowiada za sekwencyjny dostęp do elementów poprzez przeciążenie operatorów inkrementacji i wyłuskania.
4. **Klasa `NodeFactory` (Wzorzec Fabryki):** Klasa pomocnicza zawierająca metodę statyczną odpowiedzialną za bezpieczne powoływanie do życia nowych instancji węzłów.

Poniższy schemat tekstowy ilustruje powiązania pomiędzy strukturami w projekcie:



Rys. 3.1. Schemat połączenia klas

3.4. Sposób użycia systemu kontroli wersji (Git)

W toku prac nad projektem wdrożono profesjonalny model Gałęzi (*Branching Model*), co pozwala na bezpieczne wprowadzanie modyfikacji:

1. **Główna gałąź (`main`):** Zawiera wyłącznie przetestowany, w pełni stabilny kod źródłowy oraz gotową aplikację.

2. **Gałęzie testowe/robocze (np. `testowy-branch`):** Tworzone poleceniem `git checkout -b nazwa_brancha` na potrzeby implementacji nowych wzorców (Iterator, Fabryka) lub modyfikacji algorytmów.

3. **Synchronizacja zmian:** Pozytywny wynik testów lokalnych skutkuje scaleniem kodu za pomocą polecenia `git merge` z gałęzią główną, a następnie przesłaniem zmian na zdalne repozytorium GitHub za pomocą `git push origin main`.

4. Implementacja

4.1. Omówienie wybranych fragmentów kodu źródłowego

Implementacja projektu została podzielona na modularne pliki nagłówkowe oraz pliki źródłowe, co zwiększa czytelność kodu oraz ułatwia jego konserwację. Poniżej przedstawiono kluczowe fragmenty oprogramowania realizujące najważniejsze założenia algorytmiczne i architektoniczne.

4.1.1. Bezpieczne wstawianie elementu pod wskazany indeks

Jedną z bardziej złożonych operacji w liście dwukierunkowej jest wstawienie elementu w środek struktury. Wymaga to precyzyjnego przepięcia wskaźników zarówno dla nowego węzła, jak i jego bezpośrednich sąsiadów:

```
void LinkedList::addAtIndex(int index, int value) {
    if (index < 0 || index > size) {
        std::cout << "Nieprawidlowy indeks!\n";
        return;
    }
    if (index == 0) { addFront(value); return; }
    if (index == size) { addBack(value); return; }

    Node* current = head;
    for (int i = 0; i < index; i++) {
        current = current->next;
    }

    Node* newNode = NodeFactory::createNode(value); // U ycie
    wzorca Fabryki
    Node* prevNode = current->prev;

    prevNode->next = newNode;
    newNode->prev = prevNode;
    newNode->next = current;
    current->prev = newNode;

    size++;
}
```

Listing 2. Metoda addAtIndex

4.1.2. Implementacja Wzorca Iteratora

Wzorzec Iteratora[5] pozwala na sekwencyjny dostęp do elementów kolekcji bez ujawniania jej wewnętrznej budowy. Poniższy fragment prezentuje przeciążenie operatorów inkrementacji oraz wyłuskania:

```
class ListIterator {
private:
    Node* current;
public:
    explicit ListIterator(Node* ptr) : current(ptr) {}

    int& operator*() const {
        return current->data;
    }

    ListIterator& operator++() {
        if (current != nullptr) {
            current = current->next;
        }
        return *this;
    }

    bool operator!=(const ListIterator& other) const {
        return current != other.current;
    }
};
```

Listing 3. Wzorzec projektowy - iterator

4.2. Opis uzyskanych wyników i działania programu

W wyniku poprawnej kompilacji i uruchomienia programu (zarówno za pomocą skryptu konsolowego MSVC `cl.exe`, jak i przez środowisko Visual Studio) otrzymano w pełni stabilną aplikację konsolową. Uzyskane rezultaty obejmują:

1. **Dynamiczne zarządzanie pamięcią:** Program poprawnie alokuje pamięć dla nowych węzłów w stercie oraz – dzięki zaimplementowanemu destruktorowi i metodzie `clear()` – zwalnia ją całkowicie w momencie zamknięcia, nie powodując wycieków pamięci (*memory leaks*).

2. **Poprawność strukturalna:** Operacje wstawiania oraz usuwania elementów na początku, końcu oraz pod wskazanym indeksem zachowują spójność wskaźników `next` oraz `prev`.

3. **Efektywność iteracji:** Zastosowanie dedykowanego iteratora umożliwiło czytelne i bezpieczne przeglądanie zawartości struktury danych, co zbliża implementację do standardów biblioteki STL języka C++.

4. **Generowanie dokumentacji:** Użycie komentarzy zgodnych ze standardem Doxygen pozwoliło na automatyczne wyczyszczenie struktury kodu i przygotowanie kompletnego raportu API projektu.

5. Wnioski

Przeprowadzony projekt pozwolił na praktyczne zastosowanie zaawansowanych koncepcji programowania obiektowego oraz struktur danych w języku C++. Realizacja zadania dostarczyła kilku istotnych wniosków o charakterze technicznym i merytorycznym:

1. **Elastyczność i wydajność struktur dynamicznych:** Dwukierunkowa lista wiązana udowodniła swoją przydatność w scenariuszach wymagających częstych operacji modyfikacji zbioru danych (wstawiania i usuwania) w czasie stałym $O(1)$ po uzyskaniu dostępu do węzła, eliminując narzut związany z realokacją tablic dynamicznych.

2. **Zalety wzorców projektowych:** Wdrożenie wzorca *Iterator* oraz *Fabryka* znacząco podniosło czytelność kodu oraz jego modularność. Wzorzec fabryki pozwolił na scentralizowanie procesu tworzenia obiektów, natomiast iterator uniezależnił logikę przeglądania elementów od wewnętrznej reprezentacji wskaźników listy, co stanowi dobrą praktykę inżynieryjną zgodną ze standardami biblioteki STL.

3. **Rola narzędzi wspierających:** Wykorzystanie nowoczesnego środowiska programistycznego (Visual Studio z kompilatorem MSVC), systemu kontroli wersji Git oraz generatora Doxygen pozwoliło na sprawne zarządzanie kodem, bezpieczne testowanie eksperymentalnych rozwiązań na oddzielnych gałęziach oraz automatyzację procesu tworzenia dokumentacji technicznej.

4. **Znaczenie zarządzania pamięcią:** Ręczne zarządzanie pamięcią w języku C++ wymaga szczególnej dyscypliny. Implementacja jawnego destruktora oraz metody czyszczącej zapobiegła potencjalnym wyciekom pamięci (*memory leaks*), co jest kluczowe przy pracy ze strukturami opartymi na dynamicznych wskaźnikach.

Podsumowując, cel projektu został w pełni osiągnięty. Powstała aplikacja jest stabilna, poprawna algorytmicznie i dobrze udokumentowana, co stanowi solidną podstawę do dalszego rozbudowywania programu o bardziej zaawansowane mechanizmy lub uniwersalne szablony klas (*templates*).

Bibliografia

[1] Bjarne Stroustrup. *Język C++. Kompendium wiedzy*. Gliwice: Helion, 2014.

[2] *Strona główna systemu kontroli wersji Git*. URL: https://git-scm.com (dostęp 30.03.2026).

[3] *Dokumentacja platformy GitHub*. URL: https://docs.github.com (dostęp 23.10.2026).

[4] *Strona oficjalna generatora dokumentacji Doxygen*. URL: https://www.doxygen.nl (dostęp 23.10.2026).

[5] Erich Gamma i in. *Wzorce projektowe. Elementy oprogramowania obiektowego wielokrotnego użytku*. Gliwice: Helion, 2010.

Spis rysunków

1.1. Schemat powiązań w liście dwukierunkowej 3
1.2. Wynik działania testów listy (operacje wyświetlania, dodawania, usuwania) 5
2.1. Przykładowy algorytm LRU 6
2.2. Strona HTML wygenerowana przez program 8
3.1. Schemat połączenia klas 10

Spis tabel

1.1. Złożoność obliczeniowa operacji 3

Spis listingów

1. Definicja struktury węzła Node 4
2. Metoda addAtIndex 11
3. Wzorzec projektowy - iterator 12